Master's thesis

E-graphs modulo theories

with applications to AC

Sofia Brookie
sofia.ma@protonmail.com

Supervisor: Nicholas Smallbone
Examiner: David Sands

June 1, 2026

Contents

Chapter 1

Abstract

E-graphs are a data structure for equational theorem proving, and which provide the basis for *equality saturation* (section FiXme: TODO), a method to automatically derive equational consequences from a starting set of terms, equations, and identities. E-graphs have generated a lot of interest since the publication of the Egg library developed by Willsey et al. and their corresponding paper ([15]).

However, E-graphs struggle with a combinatorial explosion when reasoning about certain theories. A notorious case is that of AC theories, those containing associative and commutative operators: the space complexity of saturation is doubly-exponential in the size of the input in the worst case, as shown in section 3.

Can equality saturation for AC theories be done in a more efficient way? There have been several proposals for extending E-graphs to *E-graphs modulo theories* (EMTs) ([16], [10], [11]). These proposals suggest integrating a *theory-specific* reasoning method for AC together with the general mechanisms of E-graphs and equality saturation in order to efficiently deal with AC theories. The hope is that specific methods for AC theories can avoid the inefficiencies in using the general equality saturation mechanism for the AC identities.

EMTs are inspired by the success of theory-specific methods in other automated reasoning tasks, such as in *Satisfiability Modulo Theories* solvers (SMT solvers), and the merits of a theory-specific approach in automated reasoning are advocated for in Shankar [12]

We will develop the idea of EMTs and show that it *is* possible to reason with AC theories and a variety of closely related theories in a way that avoids the worst inefficiencies of plain E-graphs.

In section FiXme: TODO, we also provide a formal definition of EMTs that we hope can provide the basis for future work on integrating other theories in a similar manner.

We then, in section 6, discuss an implementation of EMTs and benchmark it against pure E-graph reasoning on a variety of equational reasoning tasks (6.3).

Central to our notion of a 'good' theory for an EMT is the necessity of a solver for the word problem over the theory, which we justify in section FiXme: TODO. This gives us a lower bound on EMTs for AC: the word problem for AC requires in the worst case exponential space and doubly-exponential time. This is an improvement over the doubly-exponential space upper bound for E-graphs, but highlights that concrete efficiency gains derive from the potential for typical problems to be solvable efficiently. Our solver for AC and polynomials is builds upon Buchberger's algorithm, which has received significant effort to achieve good common-case performance.

On the other hand, for the theory ACUN (AC with unit and negation, aka the theory of Abelian groups) or the theory of vector spaces, we have polynomial time bounds: we can use algorithms for Hermite normal form (for Abelian groups) and Gaussian elimination (for vector spaces).

Chapter 2

Introduction to E-graphs

2.1 Equational theorem proving

The problem we are interested in solving is to take a set of equations and figure out the set of consequences of those equations. There are some different cases to consider:

1. We know the goal equation, so we want to either *validate* that it follows from the input equations or determine that it doesn't follow

2. We don't know the goal equation, and we want to explore some interesting equations that follow from the input equations

E-graphs can be used for both, but they particular shine in the second case, as they are not fundamentally 'goal-directed'.

Let's consider an example.

$$f(x, y) = f(y, x) \tag{2.1}$$
$$f(1, 1) = 2 \tag{2.2}$$
$$f(2, 1) = 3 \tag{2.3}$$

Here, the first equation is understood to hold for any values of x and y. We will call such an equation an *identity*: really, it stands for the universally quantified formula $\forall xy.\ f(x, y) = f(y, x)$. We will then reserve the term 'equation' for *ground* equations: one without such quantifiers.

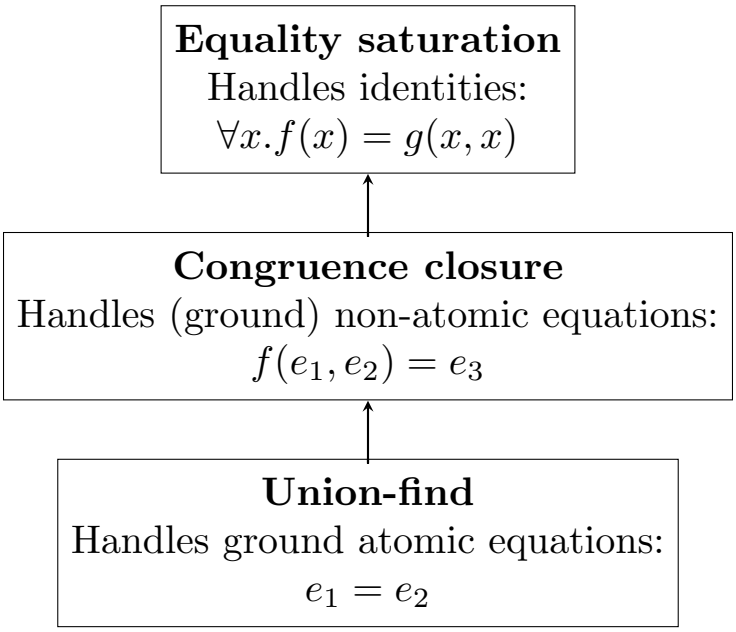
Now, here is one consequence of the above equations:

$$
\begin{aligned}
& f(1, f(1,1)) \\
&\quad = f(f(1,1),1) && \text{by 2.1} \\
&\quad = f(2,1) && \text{by 2.2} \\
&\quad = 3 && \text{by 2.3}
\end{aligned}
$$

E-graphs are a data structure on which we can run *equality saturation* to discover the above equational consequence automatically, alongside other consequences.

Applications of equality saturation include optimization in compilers, decompilation, and program synthesis ([15]) FiXme: need ref for program synthesis. Also theorem proving?.

2.2 Union-find, congruence closure, and equality saturation



Describing E-graphs can be done in 3 layers. First, there is the union-find structure, which is a simple solver for the theory of atomic equations: given a set of *atoms* $e_1, e_2, \ldots$, we can union two of them to set them equal, and we can find a representative of any particular atom. The representative is unique, so we can simply compare representatives for equality to determine if the atoms they represent are equivalent.

The data structure for this can be thought of as presenting a rewriting system among atoms: for instance,

$$
\begin{aligned}
e_1 &\to e_1 \\
e_2 &\to e_2 \\
e_3 &\to e_3 \\
e_4 &\to e_4
\end{aligned}
$$

is the trivial equivalence relation on 4 atoms. Unioning e_1 and e_2 gives

$$
\begin{aligned}
e_1 &\to \underline{e_2} \\
e_2 &\to e_2 \\
e_3 &\to e_3 \\
e_4 &\to e_4
\end{aligned}
$$

where we have (arbitrarily) chosen e_2 as the new representative.

If we union e_3 with e_4, we get

$$
\begin{aligned}
e_1 &\to e_2 \\
e_2 &\to e_2 \\
e_3 &\to \underline{e_4} \\
e_4 &\to e_4
\end{aligned}
$$

Now, if we union e_3 and e_2, we could get

$$
\begin{aligned}
e_1 &\to e_2 \\
e_2 &\to \underline{e_4} \\
e_3 &\to e_4 \\
e_4 &\to e_4
\end{aligned}
$$

Note that we do a lookup before performing the union, so that we union the two *representatives* (e_4 of e_3 and e_2 of e_2) together.

Now it takes two steps to rewrite e_1 to e_4. It is possible to either eagerly canonize the rewrite system on update, setting e_1 to rewrite to e_4, or do it lazily (compressing such multi-step rewrites on lookup), which is asymptotically more efficient in general. However, these efficiency gains are hard to realize given the congruence closure structure we will build on top of the union-find structure.

On top of union-find, we can build a congruence-closure structure. To handle non-atomic terms, we have to consider function symbols applied to atoms and other terms (like $f(g(e_1), e_2)$), and we need to respect *congruence*: if $s_i = t_i$, then we must conclude $f(s_1, \ldots) = f(t_1, \ldots)$.

To handle congruence, we will break terms down into *layers*. For instance, to represent $f(g(e_1), e_2)$ we will assign a new constant e_3 for $g(e_1)$, and then assign a constant e_4 for $f(e_3, e_2)$. This way, all congruence checks only need to apply to 'flat' terms: if $e_i = d_i$ then we conclude $f(e_1, \ldots) = f(d_1, \ldots)$.

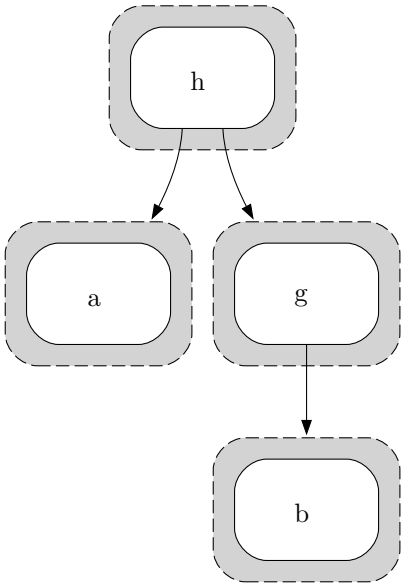
This motivates the E-graph data structure, and the congruence closure algorithm that restores the congruence invariant. Computing the congruence closure is also known as 'rebuilding' the E-graph.

2.2.1 E-graphs for congruence-closure

As an example, we will compute the congruence closure of the following set of equations:

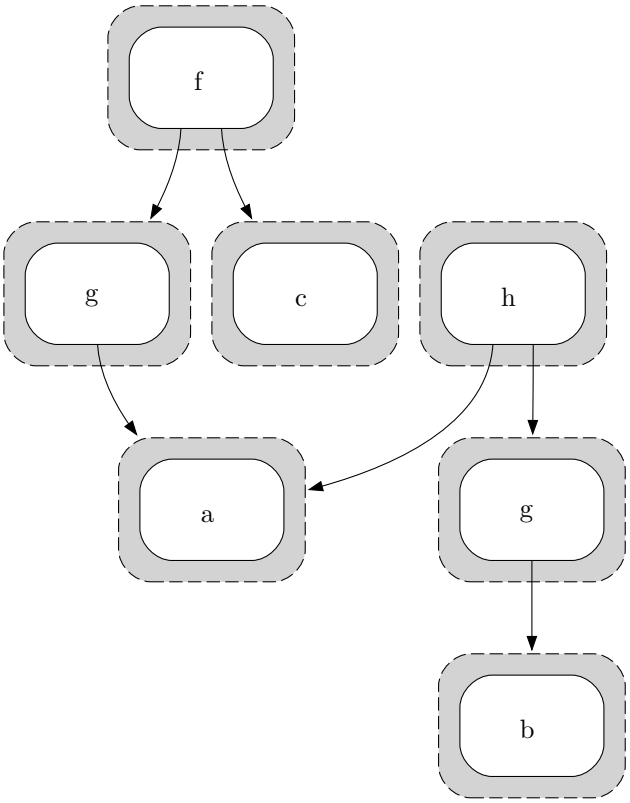
$$f(g(a), c) = h(a, g(b)) \tag{2.4}$$
$$h(g(c), b) = h(a, g(c)) \tag{2.5}$$
$$g(b) = g(c) \tag{2.6}$$
$$f(g(a), c) = g(b) \tag{2.7}$$

As a first step, we represent the term $h(a, g(b))$ from 2.4.

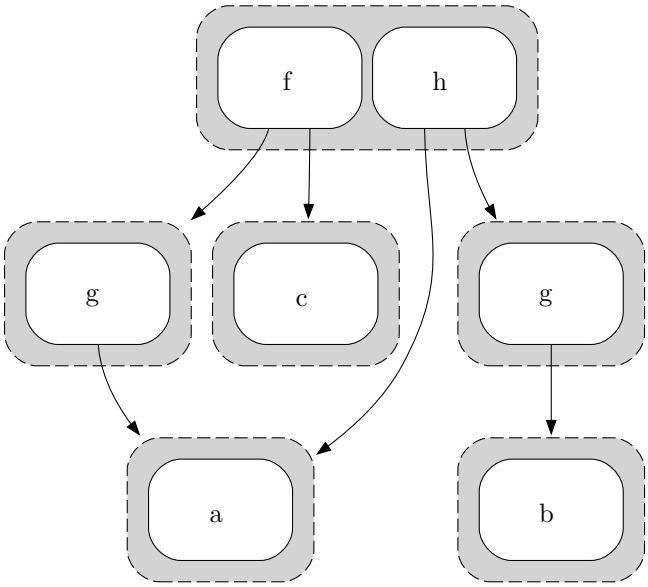


Notice how we have flattened the term: the three levels of nesting become pointers between the three layers of our graph.

Then we will add the other term $f(g(a), c)$ from 2.4.

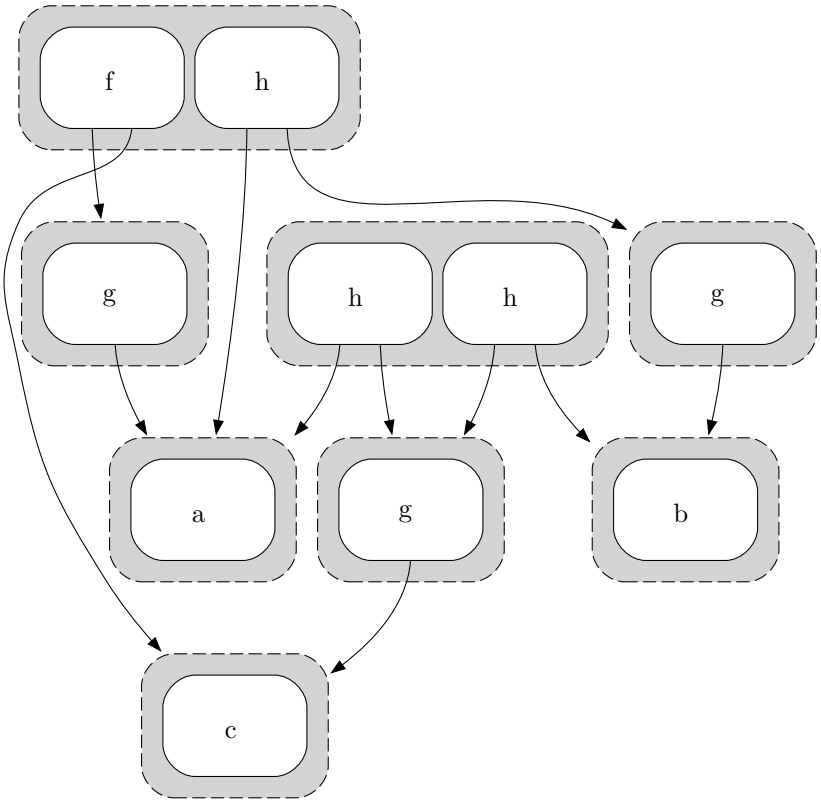


Now, as a final step, we need to *union* the two top nodes of both sides of the equation:

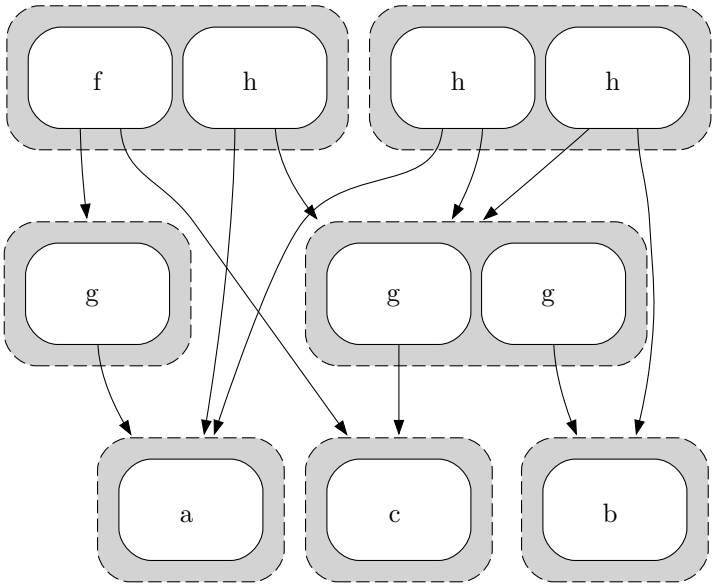


We call the outer nodes *E-classes* (equivalence classes), while the inner nodes are *E-nodes*. E-classes can contain multiple E-nodes. The arrows point from E-nodes to E-classes.

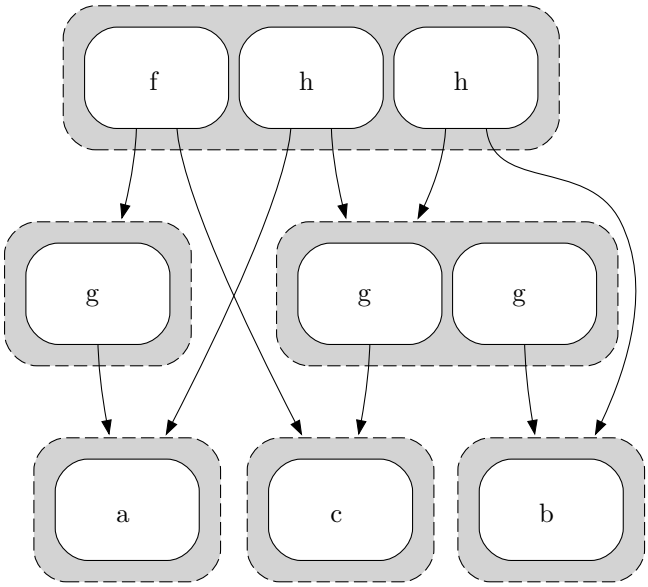
Next we move on to 2.5. This time, notice that the subterm $g(c)$ is used twice, and we *share* this in the E-graph, indicated by two arrows pointing to the E-class for $g(c)$.



Now we handle 2.6, which doesn't add any new E-nodes in the graph. Instead, it merges the separate E-classes representing $g(b)$ and $g(c)$ into a single E-node.

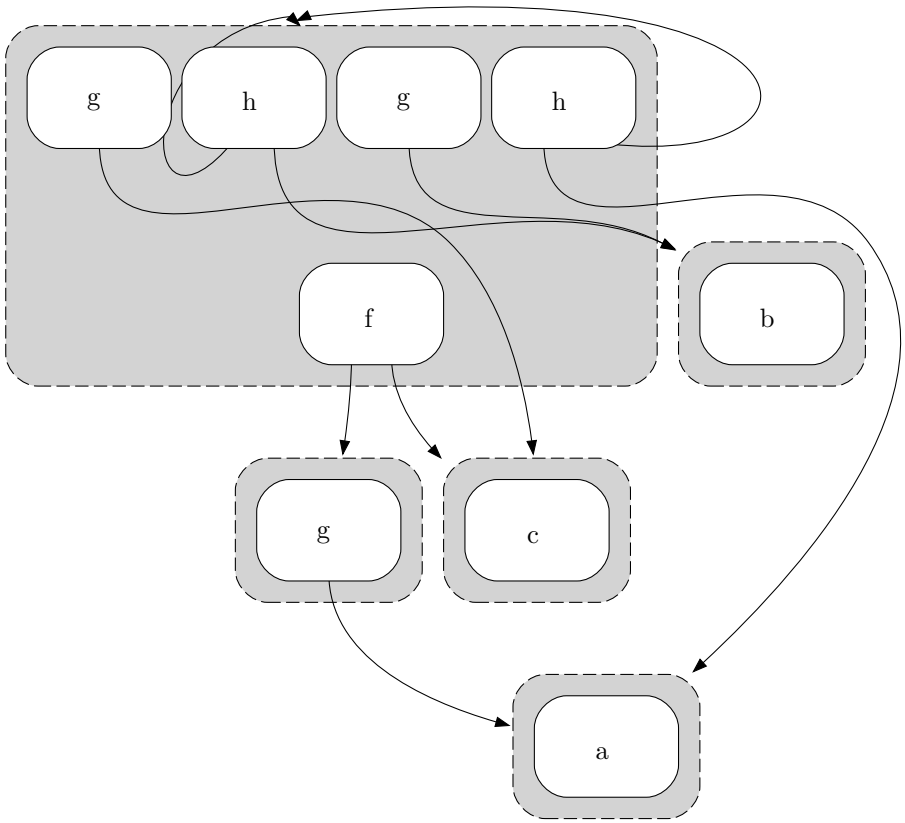


Now notice that we have two instances of an h with the same arguments: $h(a, g(b))$ and $h(a, g(c)$ have become equal by congruence, but they are located in two different E-classes. So, after unioning, we need to *propagate* by continuing to union things that have become equal. This gives us the simpler E-graph



Now we handle 2.7. This time, it will create a *cycle* in the E-graph: an E-node pointing to its own E-class. A simple equation that generates a cycle is $g(d) = d$, which gives rise to the infinite set of equations $g^n(d) = d$ by congruence.

In general, we might have cycles going through multiple intermediary nodes: $f(\ldots g(\ldots e_1 \ldots) \ldots) = e_1$. Cycles become very important in the analysis of how theories fare with E-graphs under equality saturation (section FiXme: TODO).



Handling congruence closure in E-graphs with cycles is not difficult: we can just propagate until nothing changes. There is a finite upper bound on propagation: at most, we could discover that every E-node is equal to every other. Every step moves closer to that bound by merging some E-nodes. Thus, this fix-point algorithm clearly terminates.

Equality saturation

Suppose instead of the equations $h(g(c), b) = h(a, g(c))$ in the above example we had the identity $\forall x.\ h(a, x) = h(x, b)$. To do equality saturation with this identity, we would *match* the left-hand-side $h(a, x)$ against our E-graph, which would find the match $h(a, g(b))$, corresponding to the substitution $x = g(b)$. Then, we apply the same substitution to the right hand side, thus finding the *equations* $h(a, g(b)) = h(g(b), b)$. Inserting this equation into our E-graph and using congruence closure would then yield the above E-graph.

Equalities are symmetric, so we need to match the right-hand-side and add the substituted left-hand-side in general equality saturation.

So, in general, to handle identities, we use the following procedure:

1. Add every ground equation to the E-graph
2. Equality saturation outer loop:
   (a) (E-match) Match any side of an identity to a ground term represented by the E-graph
   (b) (Insert) the other side: using the substitution from the previous step, construct the term resulting from applying the substitution to the other side of the identity. If it is not already represented by the E-graph, add it to the E-graph.
   (c) (Merge) the E-classes representing the two sides from above
   (d) Repeat

If the fixed point is reached, we call the graph **saturated**.

In practical applications such as compilers, we often have a final step to *extract* some minimal term equal to the starting term. Extraction is covered later FiXme: TODO

Equality saturation: limitations

When both substituted terms from E-matching an identity are already represented by the graph, we call the merge a *proper merge*. Unfortunately, proper merges aren't enough. If we consider the associative identity $\forall xyz.\ x+(y+z) = (x+y)+z$, and we begin with the two terms $w+(x+(y+z))$ and $((w+x)+y)+z$, we would like to prove that these terms are equal. However, the attempt to prove even the first step, that $w + (x + (y + z)) = (w + x) + (y + z)$, founders. Our representation includes no equivalence class to represent $(w + x) + (y + z)$, and so there are no equivalence classes to merge.

The 'solution' is to include the possibility of **inserting** new classes during the procedure. We call a mix of insertion and a merge an **improper merge**.

Insertion has one key problem: we now have no guarantees the process of merging will terminate. With only proper merges, there are a finite set of input terms and thus a finite bound on the equivalence relation we maintain. Thus, eventually we will get to a point where no merge changes the equivalence relation, and we have reached saturation. Introducing new classes for inserted terms now means that saturation is no longer guaranteed to be reached at any point, and worse, the strategy for insertion and merging now can affect whether

or not saturation is reached.

To even get a *semi-decision procedure*, where we aim to prove a specific equality if it exists and can fail otherwise, we must ensure that the insertion and merging processes are sufficiently *fair* (in the sense of 'fair' as used in concurrent programming): we must have some strategy to ensure every possible term that could be inserted *does* get inserted eventually, or we might get stuck inserting other terms infinitely while nonetheless not making progress on the goal.

In some sense, this is to be expected, as even for just equational theorem proving, there are theories which are known to be undecidable. However, even when the process of E-matching, inserting and merging doesn't terminate, we can consider the infinite fixed-point of this process as the 'semantics' of our E-graph [13], and this will be what we will preserve in the shift to EMTs.

2.2.2 Aside: variations on equality saturation

We can do several matches/insertions/merges in between rebuilding; it is sufficient that rebuilding is *eventually* run after changes to the E-graph [15].

There are plenty of extensions to the outer loop that include things such as conditional reasoning (along the lines of: if $a = b$, then assert that $c = d$ by merging classes) or E-class analyses ([15]).

Chapter 3

Egraphs modulo AC

One common pair of identities we want to work with is associativity

$$\forall xyz.\ f(x, f(y, z)) = f(f(x, y), z) \tag{3.1}$$

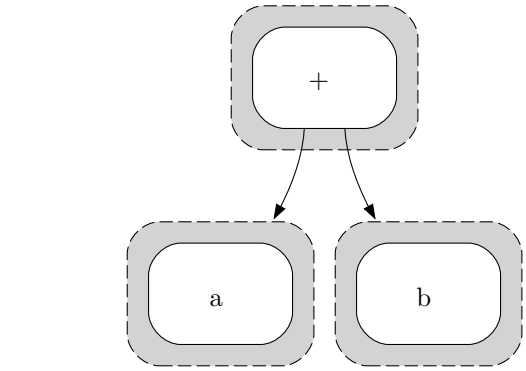
and commutativity

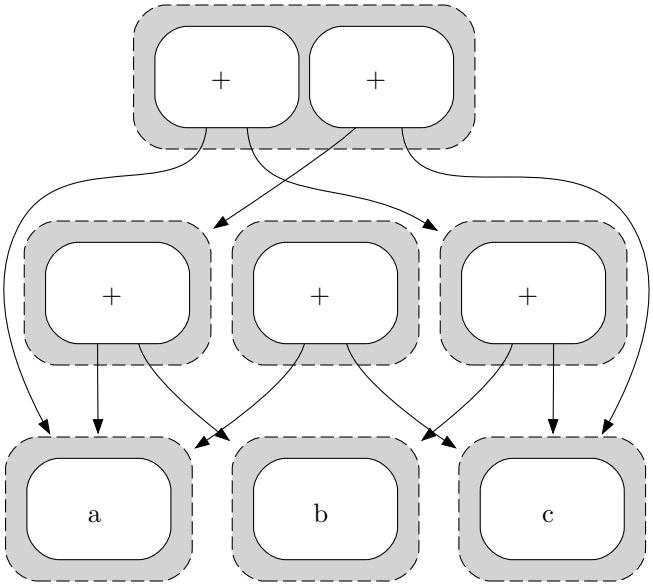
$$\forall xy.\ f(x, y) = f(y, x) \tag{3.2}$$

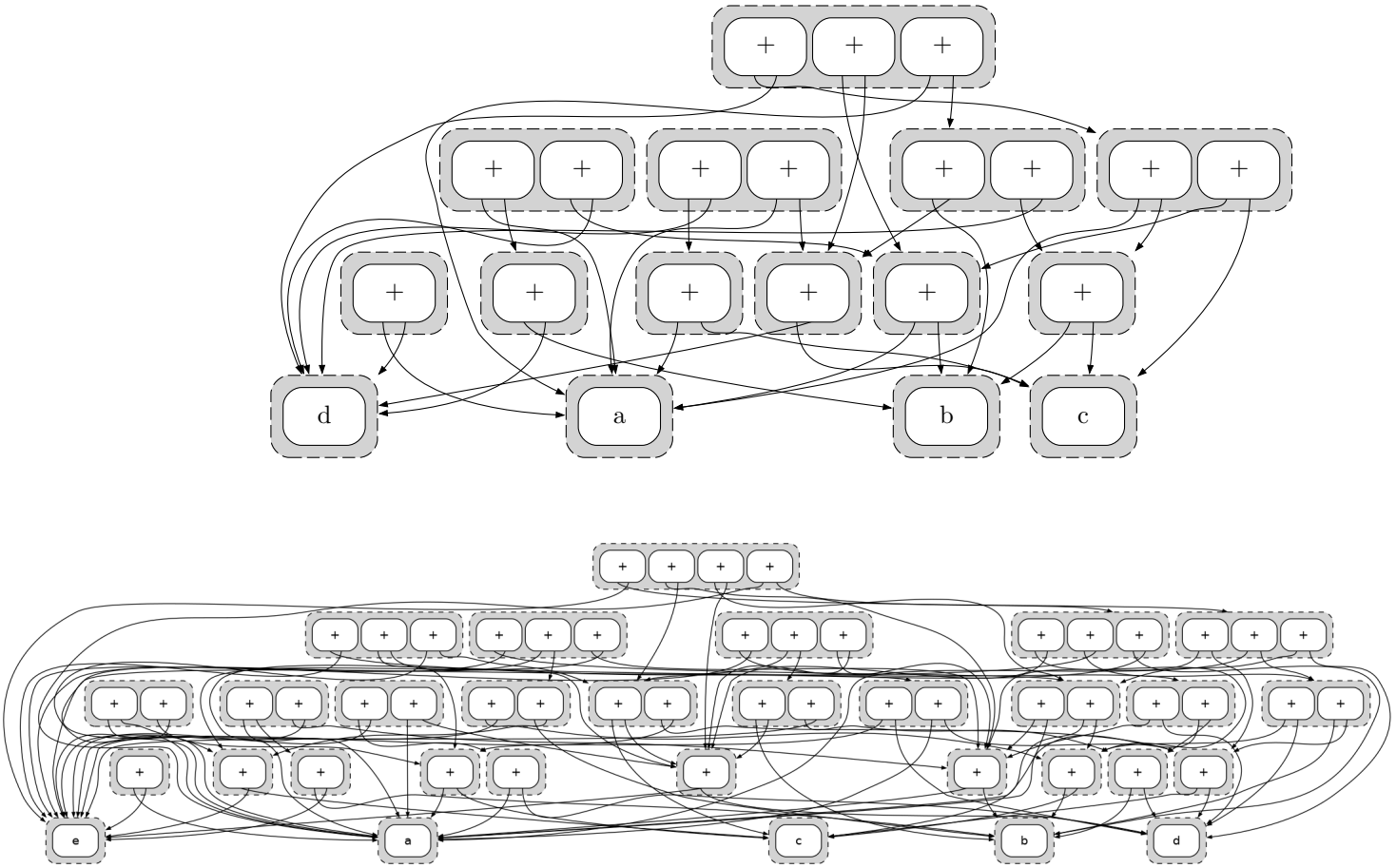
We refer to this combination as AC

A common AC operator is $+$, and we will use that as a generic name for an AC operator.

Unfortunately, equality saturation handles these identities rather poorly. Consider as a simple case the terms of length n over some constant terms k_n $t_n = a + (b + (...k_n))$. Then we can see that there is a significant blowup in the size of the saturated graph as we increase n:


+
a
b


+
+
+
+
+
a
b
c



It's relatively easy to see why: the top E-class has $2^n - 1$ E-nodes, one for each way to partition the set of atoms into 2 nonempty parts, so the E-nodes in the top class already have $2^n - 1$ E-classes as children. And then each child E-class representing the sum of k atoms has its own 2^k enodes, etc. So, there are $O(2^n)$ E-nodes for each of $O(2^n)$ E-classes, giving a doubly-exponentially $O(2^{2^n})$ sized E-graph.

3.1 Handling AC with EMTs: a first example

The difficult with plain E-graphs motivates developing a notion of *E-graphs modulo theories*, where we work modulo AC (the theory we will focus on now)

We will use an example problem: besides the AC operator $+$, we introduce a function symbol f and the identity $\forall xy.\ f(x+y)+1 = f(x)+f(y)$. We also allow integer constants k_i, and treat integer arithmetic as an identity: $\forall xy.\ k_i + k_j =$

k_{i+j}. Indeed, we will write integer constants directly.

We begin with the term $f(a) + (f(b) + f(c))$. We would like to discover that this input term is equal to $2 + f(a + (b + c))$, using the reasoning

$$
\begin{aligned}
&f(a) + (f(b) + f(c)) = \\
&f(a) + (f(b + c) + 1) = \\
&(f(a) + f(b + c)) + 1 = \\
&(f(a + (b + c)) + 1) + 1 = \\
&f(a + (b + c)) + (1 + 1) = \\
&f(a + (b + c)) + 2 = \\
&2 + f(a + (b + c))
\end{aligned}
$$

We will present a pictorial overview of how we might solve the above problem using equality saturation on EMTs rather than E-graphs.

The natural first step is to simply not add any different representations of terms that are equal up to AC; thus, nodes in the E-graph for AC operators will be variadic and we will consider their children unordered. This is the key idea from prior work ([16], [10], [11]).

Figure 1 begins with the variadic AC-term $f(a) + f(b) + f(c)$. We omit the labels of equivalence classes.

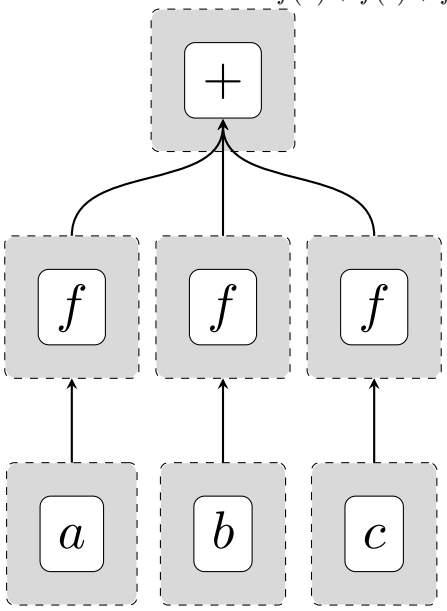
Now, in order to *match* the left-hand side of $f(x) + f(y) = f(x + y) + 1$, we need to do *matching modulo theories*, which will tell us of six matches: $(x, y) \in \{(a, b), (a, c), (b, c), (b, a), (c, a), (c, b)\}$. Then we will add the six substituted forms of $f(x + y) = 1$. Here, we will want to *insert modulo theories*, in a way that keeps the E-graph from representing multiple AC-equivalent terms. The arrows in Figure 2 have become crowded, but top E-class now represents the original term, and $f(a + b) + f(c) + 1$ and its variants with a, b, c permuted.

Now, we once again try to match $f(x) + f(y)$. The only new matches will be $(x, y) = (a + b, c)$ and its permutations, and, modulo AC, the substituted-for term $f(x + y) + 1$ will be the same for all of these. So, we just need to add a single node.

Our last step is to notice that the term $f(a + b + c) + 1 + 1$ has, up to AC, the implicit subterm $1 + 1$, which we can reduce to 2. Having reduced the implicit subterm, we no longer need the 3-part term $f(a+b+c)+1+1$, and can entirely replace it with $f(a + b + c) + 2$.

The final EMT (Figure 3) is saturated, and has 'discovered' our desired equality.

Figure 1: EMT of the term $f(a) + f(b) + f(c)$.



FiXme: Should I unfloat the preceding diagrams?

3.2 Challenges for EMT(AC)

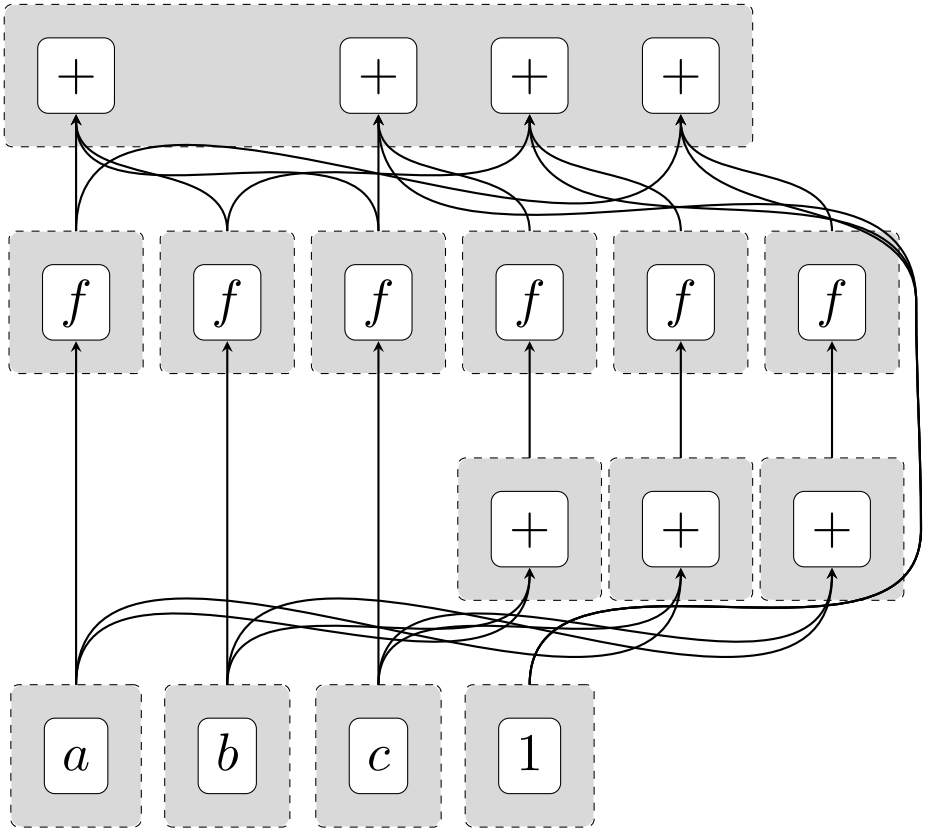
We have seen the promising aspects of EMTs for AC, but there is a significant gap in the description given so far, and we need something extra. Consider the equations

$$a + b = b \tag{3.3}$$
$$a + a = c \tag{3.4}$$
$$c + b = d \tag{3.5}$$

taken modulo associativity of $+$.

We can represent them with an E-graph as follows:

Figure 2: EMT of the term $f(a) + f(b) + f(c)$ after some steps.



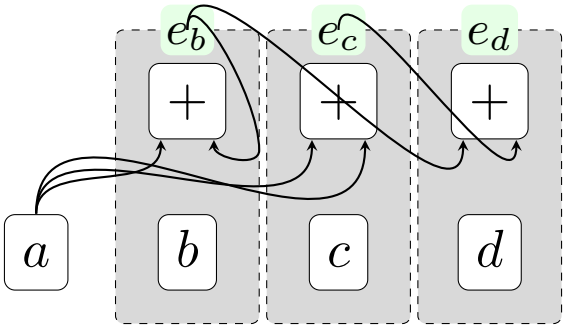
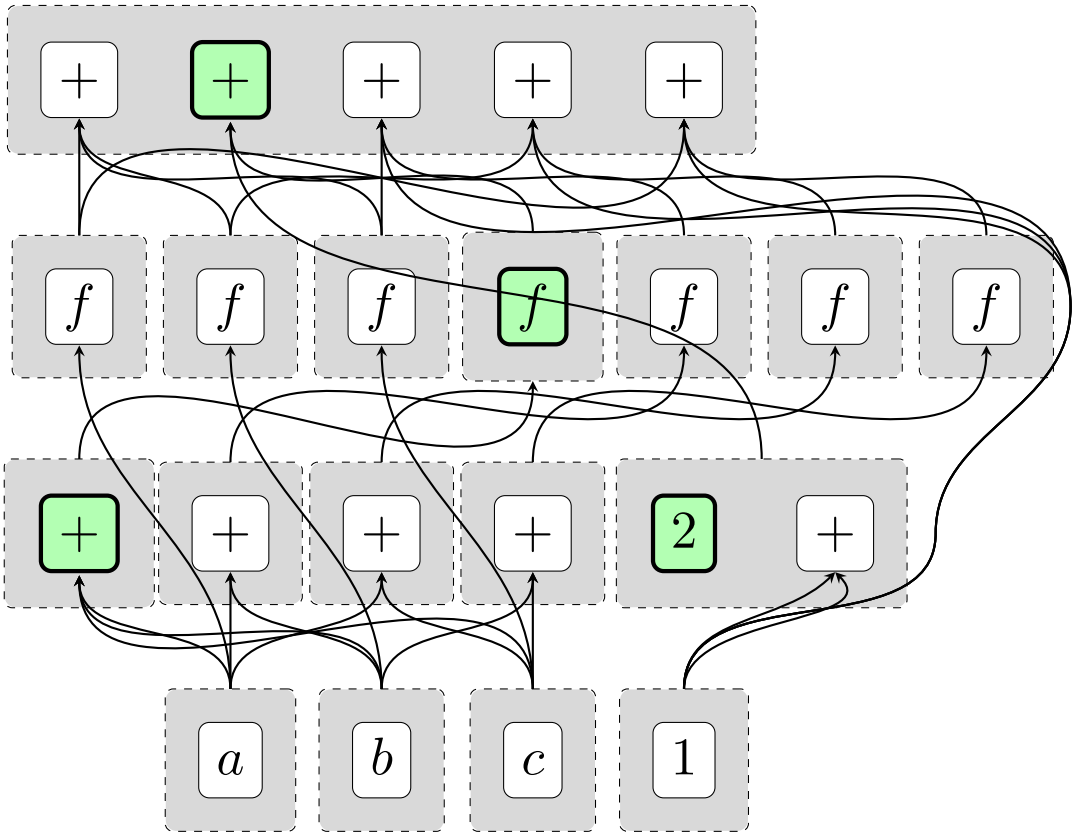


Figure 3: EMT of the term $f(a) + f(b) + f(c)$ once saturated, highlighting the desired term.



Consider the equation $b = d$, which follows from the simple deduction

$$b = a + b = a + (a + b) = (a + a) + b = c + b = d$$

However, e_b and e_d are two distinct E-classes in the E-graph! Why?

The central step in the deduction relies on two different associations of the term $a + a + b$. The association $(a + a) + b$ is correctly assigned to class e_d, while the association $a + (a + b)$ is correctly assigned to the class e_b. However, given that we want to work modulo theories, we need to discover the fact that $e_b = e_d$, or we are throwing out some of the consequences derivable from the theory.

3.3 Filling the gap

To fill the gap, we need to reason about when such an 'overlapping' term could occur, while not generating all AC-subterms like equality saturation would.

Our criteria is this: if the EMT contains two terms $t_1 = r + s$ and $t_2 = u + s$ then we need to insert the equation $t_1 + u = t_2 + r$.

This equation is a consequence, since $t_1 + u = (r + s) + u = (u + s) + r = t_2 + r$.

With our above example, we see that e_b contains the E-node $+(a, e_b)$ and e_c contains the E-node $+(a, a)$. Consequently, we insert the equation $e_b + a = e_c + e_b$. Since the right-hand-side is already assigned to e_d and the left-hand-side to e_b, this equation simply asks us to equate e_b and e_d, exactly as desired.

Let's refine the idea into something more workable. Suppose we have the variadic $+$-term $+(e_1, \ldots, e_k)$ in some E-class e, and the variadic $+$-term $+(d_1, \ldots, d_i)$ in some E-class d. Think of the arguments as multisets, so that $\{e_1, \ldots, e_k\} = E$ and similarly for D. Then let the multiset O contain each element with the minimum multiplicity it occurs in either E or D. So if $E = \{e_1, e_1, e_2, e_3\}$ and $D = \{e_1, e_3, e_4\}$, then $O = \{e_1, e_3\}$. If the overlap is empty, we can ignore it. Otherwise, construct the terms $E - O + D$ and $D - O + E$, in the obvious way, and then insert the equation $E - O + D = D - O + E$.

Is this enough? Suppose we imagine a chain of reasoning involving ground equations, the A identity, and the C identity. We can group the reasoning into 'ground' sections and the 'AC' sections. An AC section, in total, involves reassociating and commuting some term involving $+$s. So, FiXme: Write this when I'm less impatient. Proof of completeness+termination

3.4 Prospects for EMT(AC)

EMT(AC) avoids the need for equality saturation on the A and C identities by integrating these rules into the underlying E-graph. However, this has a cost: congruence closure normally is polynomial time [1] while the AC-closure step outlined above is at worst ESPACE [2] as will be discussed later. FiXme: forward ref

Moreover, while congruence closure only shrinks the graph by potentially discovering E-classes that can be merged, AC-closure can grow it, so one might want to run AC-closure only intermittently while running congruence closure much more often. This is then not that different in spirit to existing approaches to handling AC blowup in equality saturation by 'deprioritizing' the AC identities FiXme: Do I need a source here

[1] indeed, at best slightly superlinear

[2] space proportional to $2^{O(n)}$, where n is the size of the E-graph

However, there are two key advantages:

- In many practical cases, the AC-closure is small, while ordinary equality saturation will have to run to completion to be 'sure' that nothing is missed.

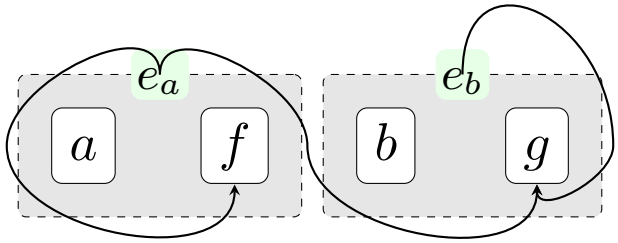
- AC-closure is much better at finding 'interesting' nodes to add to the E-graph, in the sense that the added nodes are part of some nontrivial equality $t_1 + u = t_2 + r$. Equality saturation on AC will generate many 'trivial' identities alongside such potentially useful ones.

Chapter 4

E-graphs, EMTs, and rewriting

4.1 E-graphs present a rewriting system

Consider the E-graph



We can read off a set of equations from our final E-graph:

$$\begin{aligned} a &= e_a \\ f(e_a) &= e_a \\ b &= e_b \\ g(e_a, e_b) &= e_b \end{aligned}$$

Given this resulting set of equations, we can determine if any two terms at all are equal under our initial set of equations: simply reduce the terms by treating these equations as a *rewrite system*, read from left-to-right, and see if the normal forms are the same. This is the perspective of Abstract Congruence Closure ([2]): E-graphs present a rewriting system for terms over an *extended signature*. The signature is extended in the sense that we don't just have the symbols f, a, g, etc, but also $e_a, e_b, \ldots$. In general, we introduce a new E-class e_i whenever we need to represent a subterm.

Then the congruence closure algorithm, aka E-graph rebuilding, is a matter of restoring *confluence* to this rewrite system. Confluence is the property that any maximal sequence of rewrites end up with the same result. For instance, consider

$$f(e_1, e_2) \to e_3$$
$$f(e_1, e_2) \to e_4$$

$f(e_1, e_2)$ can rewrite to both e_3 and e_4. This would violate confluence, unless we have the rewrite $e_3 \to e_4$ or $e_3 \to e_4$. Let's say that $e_3 \to e_4$. Then $f(e_1, e_2) \to e_3 \to e_4$, so the divergence $e_3 \leftarrow f(e_1, e_2) \to e_4$ is only temporary: the left rewrite is not maximal and can be extended.

Of course, confluence applied in this case is simply an instance of the congruence rule: $f(e_1, e_2)$ is in two E-classes that need to be merged by taking the union of e_3 and e_4, which introduces the rule $e_3 \to e_4$ or its converse.

4.2 E-graphs simplify confluence

E-graphs in the rewriting perspective consist of two kinds of rules: union-find rules $e_i \to e_j$ and E-node rules $f(\ldots) \to e_k$. To turn an equation $s = t$ into rules, we flatten it completely into E-node rules, and add a union-find rule when the resulting top E-nodes would be in different classes.

The advantages of this flattening are twofold. Firstly, termination[1] of the rewriting system is easy to determine: E-node rules make a term smaller, so we just need to ensure there is no infinite cycle among the union-find rules. Secondly, confluence is much easier to check for and restore.

Termination and confluence together give convergence, which means that we can apply the rules in any order to a term and get a unique normal form. If the normal form is e_i, then that term is in the E-graph and equivalent to any other

[1] in some sectors of computer science, termination is more commonly known as *strong* normalization

term that also yields e_i. We could also get, say, $f(e_1, e_2)$, which means that the term is not in the E-graph, but its two subterms are.

We will use $\to^*$ for the reflexive transitive closure of the rewriting relation.

Here are all the ways in which an E-graph-like rewriting system could fail to be confluent:

1. $e_1 \to^* e_2$ and $e_1 \to^* e_3$
2. $f(e_1, e_2, \ldots) \to^* e$ and $f(d_1, d_2, \ldots) \to^* d$, where for each i, $e_i \to^* d_i$

That's it! The first kind of case is resolved by the union-find. The second case is precisely congruence.

If, whenever we have the rule $e_i \to e_j$, we replace e_i everywhere in our rewriting system by e_j, then the confluence rule is even simpler, as we only need to check for $f(e_1, \ldots) \to e, f(d_1, \ldots) \to d$, and $e_i = d_i$ or $e_i \to d_i$. Now we don't even have to think about $\to^*$.

So, E-graphs simplify the problem of congruence closure by reducing the problem to checking for certain kinds of non-closure (local failures of confluence), resolving them, and repeating until fixpoint. In this way there is no complexity to handling deeply-nested subterms.

Such local failures of confluence are known as critical pairs[2].

4.3 Equality satuation: handling AC-critical pairs by brute force

When we want to reason about AC, we need the notion of AC-critical pair. If $e_1 + e_2 \to e_3$ and $e_4 + e_2 \to e_5$, then there is a critical term $e_1 + e_2 + e_4$, which is AC-equal to $(e_1 + e_2) + e_4$ which rewrites to $e_3 + e_4$, and also to $e_1 + (e_2 + e_4)$ which rewrites to $e_1 + e_5$. So, our critical pair is $e_3 + e_4$ and $e_1 + e_5$.

E-graphs handle AC-critical pairs the same way as critical pairs for any other theory, with equality saturation: we *ground* the AC identities by matching either side with the E-graph, and adding the equivalent other side (if it is not present) and merging the equivalence classes corresponding to either side.

Why do E-graphs fare poorly with AC? We have seen that equality saturation with AC will generate a doubly-exponential number of E-nodes.

These 'redundant' E-nodes and E-classes play a role: any of the $O(2^{2^n})$ AC-subterms of the input could be part of an AC-critical pair, and so equality

[2] They are also required to be minimal, but we gloss over this here

saturation eagerly assigns new E-classes to them, just in case. Introducing these extra E-classes and E-nodes is seemingly the best we can do in general unless we have a clever way to detect which ground instances of the

It should be noted that equality saturation does not directly add the critical pairs of terms; rather, it adds every potential overlap, and if that overlap is then part of two terms that form a critical pair, the critical pair will be resolved by applying AC-rules and unioning the relevant nodes.identities are needed in order to ensure confluence modulo those identities.

4.4 EMTs: Handling AC-critical pairs less naïvely

We don't need to add all AC-subterms to find all critical overlaps. It suffices to consider every pair of E-nodes with an AC symbol and check them for overlap.

This procedure always terminates ([2]). It is, in fact, a special case of Buchberger's algorithm for finding Gröbner bases of sets of polynomials ([14]), a well-studied algorithm with a lot of work done in optimizing it to be fast enough in practice. FiXme: I think we will have proved termination in the introductory section by this point

FiXme: I need to explain at some point that even though EqSat doesn't add the crit pairs, just adds the overlaps until it 'discovers' certain crit pairs, AC-completion can't get away with not adding the crit pairs because it is in building the complete system including crit pairs that we discover whether overlaps are required. As in, either add all overlaps (EqSat) or don't add any but resolve crit pairs until fixpoint, because that is what it takes to eliminate the overlaps from potentially appearing in an AC-deduction that is not represented in the E-graph

So, if an E-graph has E-node rules and union-find rules, EMT(AC) adds a third kind, an AC-rule.

For AC-rules, do we want to allow rules of the form $e_1 + \ldots + e_n \to d_1 + \ldots + d_k$ or restrict to simply $e_1 + \ldots + e_n \to e$? The former is closer to usual presentations of Buchberger's algorithm, while the latter is more in the spirit of E-node rules. We will refer to these as the *side-table* and *integrated* approaches. Both are valid, but there is a subtlety in the integrated approach that might be missed, while the side-table approach makes it much more obvious.

4.4.1 EMT(AC)s via a side-table

With the side-table approach, we think of the set of AC rules $e_1 + \ldots + e_n \to d_1 + \ldots + d_n$ as being a separate thing entirely to the E-graph.

On insertion, we don't add any E-node rules whenever the function symbol is

an AC symbol $+$, but generate a new E-class name e and add the AC-rule $e_1 + \ldots + e_i \to e$ when we insert the flattened term $e_1 + \ldots + e_i$.

To compute the AC-closure, we take every pair of two AC-rules $r \to s$ and $t \to u$. Consider r, s, t, u to be multisets, and define $+, -, \max, \min$ as operations lifted across them, acting on the multiplicities. Then we can define

- The overlap: $\min(r, t)$
- The critical term: $\max(r, t)$
- The critical pair: $c_1 = \max(r, t) - r + s$ and $c_2 = \max(r, t) - t + u$

Note that $\max(r, t) + \min(r, t) = r + t$, so it is also possible to write the critical pair as $c_1 = t + s - \min(r, t)$ and $c_2 = r + u - \min(r, t)$.

When the overlap is not 0, then we take the critical pair and try to rewrite both sides by existing rules until it is minimal. If it is still non-trivial, we *orient* it, according to some ordering on flat AC-terms. A common choice for Buchberger's algorithm is *grevlex* order, graded reverse-lexicographic, which is used in our implementation.

After orienting the reduced rule, we reduce every other rule with respect to it, and add it to the set of AC rules.

FiXme: This section needs a bit more detail/clarity, probably

The final ingredient is combination rules. If we were ever to have a reduced rule $e_1 \to e_2$, with 1-term sums on either side, we should instead treat this as a union-find rule, and correspondingly propagate its consequences into the union-find and E-node rules. And vice versa, whenever the union-find rules are updated, we need to update any AC-rules according to the new ordering on atoms induced by the union-find rule.

4.4.2 Integrated EMT(AC)s

Unlike the side-table approach, here we will only allow AC rules with an atomic right-hand-side, thus making them of the same form as E-node rules.

As before, we can now form critical pairs c_1, c_2 for rules with non-empty overlap. We proceed as above and reduce the critical terms as much as possible by existing rules. Then we need to create a new E-class id e and insert the rules $c_{1\text{reduced}} \to e$ and $c_{2\text{reduced}} \to e$.

One subtlety is that here we want to reduce *existing* rules by the new one. This interreduction step is a standard step in Buchberger's algorithm, but may not be obvious in the 'integrated' viewpoint, as it is a form of destructive rewriting on the E-graph.

This interreduction shrinks the size of the E-graph, so it is important for performance.

FiXme: Can we prove these approaches equivalent easily?

In the integrated viewpoint, this amounts to shrinking E-nodes when they can 'factor' through another node. For instance, if we have E-nodes $a+b+c$ in class e_1 and $b+c$ in class e_2, we ought to destructively rewrite the first to $a+e_2$. Just resolving critical pairs will lead to e_1 containing both $a+b+c$ and $a+e_2$, which is correct as far as dealing with critical pairs but redundant.

Going further, the performance of Buchberger's algorithm depends on the ordering used (FiXme: cite), something that might not be obvious with the integrated approach. It is standard to use grevlex ordering.

Chapter 5

Beyond rewriting: E-matching, E-analysis, and E-xtraction

Many uses of E-graphs don't just use them as an equational theory solver, but want to run certain analyses on the E-graph (perhaps to allow conditional rewrites) or want to extract the smallest term representing an equivalence class, according to some cost metric.

Besides that, E-matching is a core part of the equality saturation loop.

We need to generalize all of these to EMTs. Thankfully, the 'three Es' are quite similar, and can be generalized together.

5.1 E-analysis

Suppose we have, for each function symbol f of arity $\alpha(f)$, an *interpretation* of that function symbol over a domain D, so that if $d_1, \ldots, d_{\alpha(f)} \in D$, then the interpretation $[\![f]\!]$ maps $(d_1, \ldots, d_{\alpha(f)}) \in D^{\alpha(f)}$ to $[\![f]\!](d_1, \ldots, d_{\alpha(f)}) \in D$.

This interpretation gives us a basis for an E-analysis. Keep a map a from E-class names to values in D, and when we insert $f(e_1, \ldots)$ into a new E-class e, set $a(e) = [\![f]\!](a(e_1), \ldots)$.

We need the a binary operation $\sqcap : D \times D \to D$ on the domain, which is called to merge annotations when E-classes are merged. To make this process well-defined regardless of the details of the congruence-closure/rebuilding process,

we want $\sqcap$ to be associative, commutative, and idempotent.

5.2 Extraction

Suppose we have some cost function $c : T \to K$ mapping terms $t \in T$ to a totally-ordered set C. Then we can define extraction as an analysis by taking our domain to be $T \times C$, and defining

$$(t_1, c_1) \sqcap (t_2, c_2) = \textbf{if}\, c_1 > c_2\, \textbf{then}\, (t_1, c_1)\, \textbf{else}\, (t_2, c_2)$$
$$[\![f]\!]((t_1, c_1), \ldots) = (f(t_1, \ldots), c(f(t_1, \ldots)))$$

For this to be well-defined, we require that cost is monotonic: if $c(t_i) < c(s_i)$, then $c(f(t_1, \ldots)) < c(f(s_1, \ldots)$. Otherwise, lower-cost child terms could make higher-cost parent terms, so we cannot rely on a simple propagation strategy to find the overal lowest-cost term in each E-class.

5.3 E-matching

5.3.1 Compilation

Suppose we have some pattern, like $f(x, g(a, x, y), h(y))$, where x and y are variables. The first step is to compile the patterns into a *match table*, which for this example would look like

- Leaf variables: $L = \{x, y\}$
- States:

$$\begin{aligned} a &\to s_1 \\ g(s_1, \bot) &\to s_2 \\ h(\bot, \bot) &\to s_3 \\ f(s_1, s_2, s_3) &\to s_4 \end{aligned}$$

- Final state: s_4

FiXme: Talk about how this is only one join order, and there are many other ways to do this

The idea is that, thinking of the term as a tree, we can assign a state to each node in the tree. Then, our E-analysis for doing E-matching will map E-ids to the domain of sets of pairs (state, set of substitutions).

Our interpretation for each function symbol is

$$[\![f]\!](d_1, \ldots) = \{(\bot, \{l \mapsto e \ \mid l \in L\}\} \cup \text{match}_f(d_1, d_2)$$

where match_f is defined in a way dependent on the match table, taking joins across substitutions in the appropriate way when the states match with some entry.

and $\sqcap$ unions the sets of states, and when both sides of the union contain the same state, the sets of substitutions are merged.

Note that the interpretation depends on the E-class id e, so it doesn't quite fit in the framework for E-analysis from above, but this is not a significant change.[1]

5.4 The 'Three Es' for EMTs

Of course, part of our motivation for EMTs is to avoid explicitly representing nodes that would be present in the E-graph if running purely with equality saturation. Since an E-analysis is defined per-E-class, our E-analysis will not *directly* be defined for certain E-classes that simply don't exist in our EMT.

5.4.1 Representedness of terms

It is worth spelling out exactly which E-classes are represented in E-graphs that aren't in EMTs. EMTs will also represent potentially some extra for handling critical pairs that are nonetheless not (currently) used in AC-reasoning FiXme: Better terminology needed.

It is clear that equality saturation on AC will simply generate subterms-up-to-AC of existing terms. That is, if we have a multiset of atoms E, then equality saturation will represent every binary tree whose multiset of leaves is E.

Therefore, we can define the 'representation status' of an AC-term t according to the following:

- Rewrite t according to all appropriate AC-rules until reaching a normal form
- If t is now a single E-class id e, t is directly represented
- If t is now a term $e_1 + e_2 + \ldots$, then see if t is a subset of a term appearing in an AC-rule. In this case, it is indirectly represented. FiXme: We ought to only need the LHS of rewrite rules, right?

[1] It is possible to have an 'Id-counter' as part of the domain and entirely reconstruct a set of E-class ids as a pure E-analysis independently of the ones actually used in the E-graph, but this feels more like a theoretical trick than something useful. $\sqcap$ for this domain requires a lot of remapping of E-ids, for instance

- If t is not a subset of the terms in the AC-rules, it is not represented.

Note that the 'side-table' and 'integrated' approaches (FiXme: backref) differ exactly on which things are directly or indirectly represented, due to the integrated approach generating more E-ids for certain AC-terms created during completion.

5.4.2 AC-analyses

Regardless, we can now spell out what is required for a well-behaved AC-analysis. We don't want to make any distinction between directly and indirectly represented terms.

For a directly represented term, we want a variadic interpretation function for any AC-symbol $[\![+]\!](d_1, \ldots)$ that is associative and commutative, naturally. When constructing an AC E-node, we can use this interpretation to assign the analysis.

For indirectly represented terms, we can apply the interpretation function lazily: for instance, if $e_1 + e_2$ is only indirectly represented, we don't store $a(e_1 + e_2)$ in any sense, but just calculate $[\![e_1 + e_2$
$rrbracket(a(e_1), a(e_2))$ when needed. Since this term would be directly represented in an E-graph with equality saturation, we ought to consider its annotation to nonetheless not be simply undefined.

A natural worry is that we might have, say, $e_1 + e_2$ and $e_3 + e_4$, that are both indirectly represented, but that ought to be equal: that is, if they were to be directly represented, they would be assigned the same E-class. In that case, $a(e_1 + e_2)$ ought to be equal to $a(e_1 + e_2) \sqcap a(e_3 + e_4)$. But this is exactly the problem that AC-completion solves! In particular, if t_1 rewrites to $e_1 + e_2$ and t_2 rewrites to $e_3 + e_4$, and $e_1 + e_2 = e_3 + e_4$ according to the equivalence of the EMT, then $e_1 + e_2$ and $e_3 + e_4$ would be the critical pair of some critical term.

5.4.3 AC E-matching: EMTs are no panacaea

Porting our 'matching engine' from FiXme: backref to handle AC-symbols is possible, but it does force us to confront the unavoidable difficulties of AC-matching.

Let's consider a naïve matching table for a small example pattern, $f(x + x + g(y + z))$.

$$\begin{aligned} \bot + \bot &\to s_1 \\ g(s_1) &\to s_2 \\ \bot + \bot + s_2 &\to s_3 \\ f(s_3) &\to s_4 \end{aligned}$$

Now consider matching this against $f(a + b + g(a + b) + b + a)$.

$g(a + b)$ matches up to state s_2, but we have missed the commutativity of the arguments: not only is $\{x \mapsto a, y \mapsto b\}$ a valid substitution, but also $\{x \mapsto b, y \mapsto a\}$.

Going further, we are in more trouble: We have a 5-argument $+$ in our input term, but only a 3-argument rule to reach state s_3.

All of these issues can be solved by techniques we've already developed! We will think of our matching table as an AC-rewriting system, where rewrites carry out certain operations on substitutions whenever we apply them.

$a + b$ rewrites via the first rule to s_1. Since it matches the rule in two ways (up to commutativity), we generate both substitutions in state s_1.

Then $a + b + s_2 + b + a$ matches against our third rule. Here, we generate a substitution for x that is $\{x \mapsto a+b\}$: an example of a substitution that mentions a whole *term*, since the term $a + b$ is not necessarily directly representable in the E-graph. We have to filter out all AC-matches to the third rule to the ones that map the two occurences of x to AC-equal terms, as usual.

The upshot is that AC-matching on EMTs is similar to ordinary E-matching on E-graphs, in the sense that it can be broken down to atomic, E-class/E-node-at-a-time steps. However, we have to use single-level AC-matching and AC-equality instead of single-level matching and trivial equality, and our substitutions are not pure E-class ids, but 'indirectly-represented' terms.

In particular, single-level AC-matching is quite expensive: matching $x+y+z+w$ against $a + b + c + d + e$ generates $4 \cdot 5!/2 = 240$ possible substitutions.

One thing to consider in future work is that matching engine might allow patterns with certain constraints, such as $x + y + z + w, x \leq y \leq z \leq w$, to cut down the search space. In particular, a pattern like $x + y + z + w$ is likely to come up in a context where the goal is to find instances of four things added together, without caring about full space of all substitutions.

FiXme: Ref to hardness of AC-matching. But note ordinary E-matching is already NP-hard...

FiXme: Technically, before when finding overlaps etc, we are doing AC-matching, but a rather limited kind... Maybe I should spell out the algorithm. It is of course just finding multiset partitions

Chapter 6

Implementation and evaluation

We have built a proof-of-concept implementation of EMT(AC) in Haskell. We discuss some design choices and tradeoffs we made, which could be improved upon in future work, and compare equality saturation against AC-completion running on our implementation.

6.1 Signatures

We want to be somewhat generic in the signature used to build terms. FiXme: If we have discussed Σ or Σ^* at this point, point back to that discussion. Our EMT is generic over a type constructor `enode` belonging to the following typeclass

```
class (Ord (ACSymbol enode),
       Ord (Symbol enode),
       Ord (enode EId),
       Traversable enode) => Signature enode where

  type Symbol enode

  symbolOf :: enode a -> Symbol enode
  default symbolOf :: (Symbol enode ~ enode ()) =>
                      enode a -> Symbol enode
  symbolOf = void

  type ACSymbol enode

  acSymbolOf :: enode a -> Maybe (ACSymbol enode)
```

```
default acSymbolOf :: (ACSymbol enode ~ Void) =>
                enode a -> Maybe (ACSymbol enode)
acSymbolOf _ = Nothing

reconstruct :: Symbol enode -> [a] -> enode a
default reconstruct :: (Symbol enode ~ enode ()) =>
                Symbol enode -> [a] -> enode a
reconstruct s as = s & unsafePartsOf traverse .~ as

reconstructAC :: ACSymbol enode -> [a] -> enode a
default reconstructAC :: (ACSymbol enode ~ Void) =>
                ACSymbol enode -> [a] -> enode a
reconstructAC = absurd
```

In particular, `enode` is expected to be all of a `Functor`, `Foldable`, and `Traversable`. All these classes can be derived automatically with relevant language extensions, and they provide useful functionality to work with the `EId`s within an `enode EId`. Moreover, we can define the type of (non-single-layer) terms simply as `Fix enode`, or terms potentially with `EId` leaves as `Free enode EId`, using the standard `data-fix` and `free` packages.

The type family `Symbol` is defined with the intent that `Symbol enode` is isomorphic to `enode ()`. However, the default definition can be overriden to allow more compact definitions than the generic `enode ()`. Note that an enode with `()` in every argument slot is only distinguished by the function symbol, which motivates this definition.

The type family `ACSymbol` is intended to capture the subset of AC function symbols, and comes with partial function `acSymbolOf` to determine if an E-node is headed by such a symbol. Unlike `Symbol`, AC-symbols are treated as inherently variadic. The default is that no symbols are treated as AC symbols.

`reconstruct` and `reconstructAC` build a term from a function symbol and list of arguments. The default implementation of `reconstruct` is unsafe, and relies on the number of arguments to match the arity of the symbol, or it will panic. `reconstructAC` is intended to be safe as AC-symbols are meant to be variadic. `reconstruct` is not actually used anywhere currently, and is just provided for symmetry with `reconstructAC`, which is used to build new AC-terms when finding critical pairs.

Here is an example E-node type constructor implementing `Signature`:

```
data MyENode a
        = F a
        | G a a
        | Constant Int
        | Plus [a]
```

```
        | Times [a]
        deriving (Eq, Ord, Show, Functor, Foldable, Traversable)

data ACSym = PlusSym | TimesSym

instance Signature MyENode where
  type Symbol MyENode = MyENode ()
  type ACSymbol MyENode = ACSym

  -- symbolOf: default definition is fine

  acSymbolOf (Plus _) = Just PlusSym
  acSymbolOf (Times _) = Just TimesSym
  acSymbolOf _ = Nothing

  -- reconstruct: default definition is fine

  reconstructAC PlusSym as = Plus as
  reconstructAC TimesSym as = Times as
```

6.2 Implementation tradeoffs

6.2.1 Persistent data structures

We use standard persistent ordering-based data structures from the `containers` library: `IntMap`, `IntSet`, `Map`, and `Set`. Using persistent hash-based data structures from the `unordered-containers` library might be faster, but we decided not to worry about the potential constant-factor improvements. It is possible to get *asymptotic* improvements for some parts of the algorithm by switching to non-persistent data structures, such as arrays and mutable hash-tables: this removes the $\log(n)$ overhead of persistence. However, as a proof-of-concept, we decided persistent structures are somewhat easier to work with as they don't require one to thread unique mutable instances everywhere.

6.2.2 Binarization and incremental congruence closure

Downey-Sethi-Tarjan's algorithm for congruence closure [4] improves upon other algorithms such as Nelson-Oppen's algorithm [8] by *binarizing* the function symbols. For instance, given a ternary symbol f where we might insert E-nodes of the form $f(e_1, e_2, e_3)$, we can instead replace it with a two symbols f_2 and π so that we insert the *term* $f_2(e_1, \pi(e_2, e_3))$ instead. This generates multiple E-nodes per original E-node, but it restricts the arity to at most 2, which makes certain kinds of reverse lookup fast: for each E-class id e, it can either occur in the left part of an E-node $h(e, --)$ or in the right part $h(--, e)$, for which we only need to store one other E-class id and symbol.

Such reverse lookups are important for making congruence closure incremental: when we union e with e', we want to lookup the *uses* of e and replace them with e', possibly discovering more unions to carry out in the process.

The egg library [15] doesn't use either binarization or indeed any sort of incrementality in propagating equalities, instead preferring to batch the congruence closure computation into big 'rebuilding' steps between multiple steps of equality saturation.

Our library does calculate congruence closure incrementally, but we decided for simplicity to omit the binarization trick for congruence closure. Implementing it adds some complexity, and it seems hard to make a form of binarization that works well for the AC part of the system.

6.2.3 Union-find laziness

The union-find structure can achieve an amortized time complexity of $O(n\alpha(n))$ for n union and find steps, where α is the inverse Ackermann function. This is typically achieved by some degree of laziness on union: union doesn't entirely canonicalize the structure, and instead find operations mutate the structure to compress indirect paths $e_1 \to e_2 \to \ldots$ as they are discovered.

For congruence closure, and thus E-graphs, the laziness possible in the union-find seems difficult to make use of, as we need to detect the congruence $f(a_1, \ldots, b, \ldots, a_n) = f(a_1, \ldots, b, \ldots, a_n)$ once b and c have been equated; it is not enough to just 'discover' that $b = c$ when calling find on b or c or one of their equivalents, as such a find call is not guaranteed to happen at any particular point before we need to discover the congruence.

6.2.4 Non-incrementality of AC

Unlike congruence-closure, for which it is possible to incrementally restore the congruence-closure invariant with a minimal amount of work when changes are made to the E-graph, AC-closure seems hard to make incremental. For instance, suppose we have some ordering on AC-terms, and we union e_i and e_j so that $e_i \to e_j$ becomes a union-find rule. Consider an AC-rule $e_1 + e_2 + e_i + e_3 \to e_4$. We can keep track of the 'uses' of e_i, and thus we know when merging e_i we need to update this rule, but depending on the relative ordering of e_i and e_j, the ordering of the left-hand-side of this AC-rule could change dramatically.

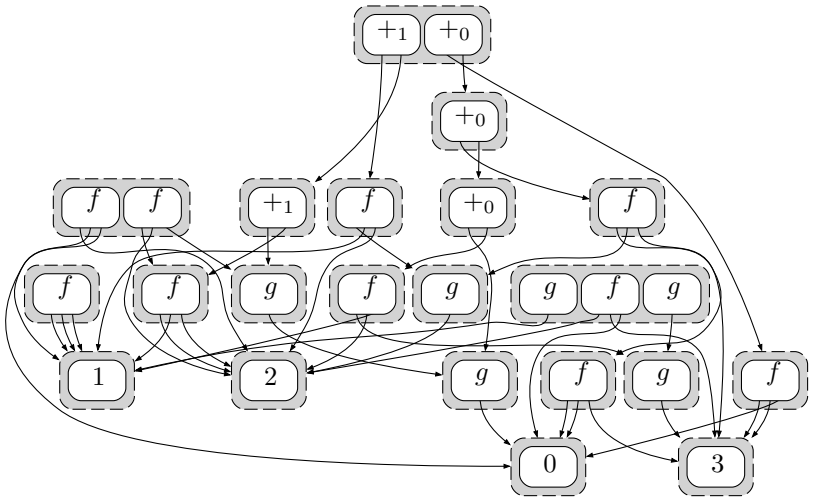
This is one of the difficulties of applying some binarization to AC-symbols: we could store the AC-rules in some kind of trie, but the structure is upended fairly arbitrarily on unions, and then we potentially have to interreduce other rules with the new one, and so forth. Thus any sort of trie-like binarized structure doesn't seem to help much when computing AC-closure.

We know FiXme: forward/backref that AC-closure is ESPACE-complete. How-

ever, this leaves room for constant-factor or even polynomial-factor improvements in the algorithmic steps of lookup, finding critical pairs, resolving critical pairs, and so forth. If certain parts of these steps can be made incremental, that is a promising path to achieving such improvements.

6.3 Evaluation

FiXme: A couple of graphs and commentary will go here after I have a few hours to code some tests



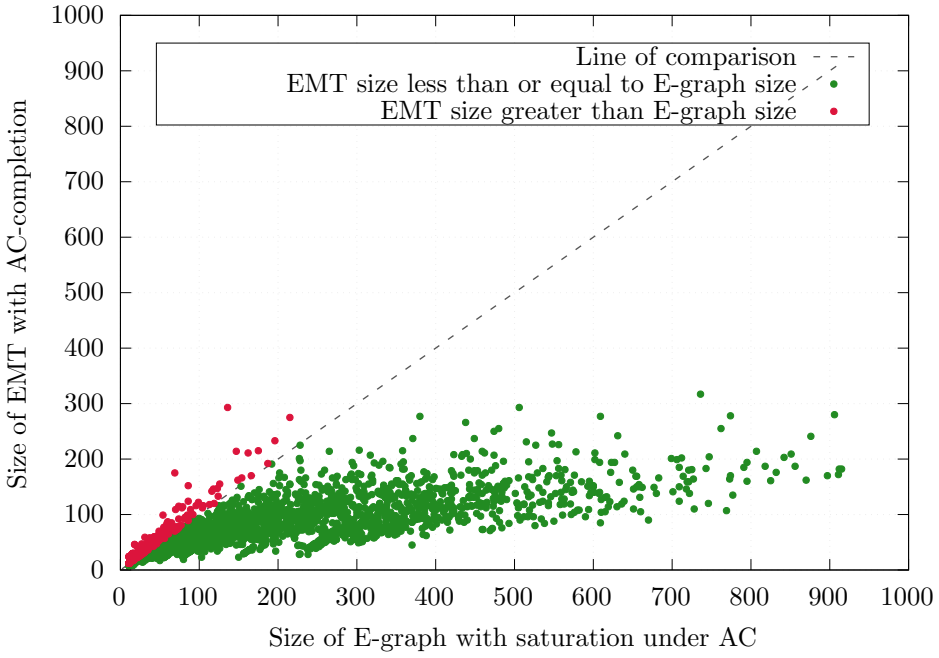


Figure 1: Size comparison between EMTs and E-graphs for random E-graphs using AC-operators

Chapter 7

EMTs for other theories

7.1 Common identities

AC is a good example of a theory which E-graphs handle poorly. However, this is not necessarily the case for every common theory, so it may not always be worth the extra complexity of using EMTs. We list some common identities which are worth analyzing:

Theory name	Abbreviation	(Function symbol, arity)	Theory
Associativity	A	$(f, 2)$	$\forall xyz.f(x, f(y, z)) = f(f(x, y), z)$
Commutativity	C	$(f, 2)$	$\forall xy.f(x, y) = f(y, z)$
Left-unitality		$(f, 2), (e, 0)$	$\forall x.f(e, x) = x$
Right-unitality		$(f, 2), (e, 0)$	$\forall x.f(x, e) = x$
Unitality	U	$(f, 2), (e, 0)$	Both of the above
Left-distributivity		$(f, 2), (g, 2)$	$\forall xyz.g(x, f(y, z)) = f(g(x, y), g(x, z))$
Right-distributivity		$(f, 2), (g, 2)$	$\forall xyz.g(f(y, z), x) = f(g(y, x), g(z, x))$
Distributivity	D	$(f, 2), (g, 2)$	Both of the above
Idempotence	I	$(f, 2)$	$\forall x.f(x, x) = x$
Inverse	N	$(f, 2), (i, 1), (e, 0)$	Unitality and $\forall x.f(i(x), x) = e$ $\forall x.f(x, i(x)) = e$
Involution	V	$(i, 1)$	$\forall x.i(i(x)) = x$

Table 7.1: Definitions of some common identities

We use the convention of naming combined theories by taking the juxtaposition of the identity names: as we have seen, AC is the theory with both A and C identities (for some function symbol), and so forth.

FiXme: cut down if some of these don't appear FiXme: also we don't need the

one-sided versions, probably

This list of identities covers a wide range of common mathematical structures, such as semigroups, monoids, groups, semirings, rings, modules, semilattices, lattices, Boolean algebras, as well as some less common ones such as bands, quasigroups, loops, and near-semirings.

Note that the condition $x \neq 0$ for x to have an inverse in a field is not formalizable in purely algebraic terms, but this list covers every other field identity. A similar observation applies to vector spaces.

7.2 How common theories fare with ordinary E-graphs

In general, using theories with equality saturation will lead to a growth in the size of the graph, as more nodes will need to be seeded to ensure that all consequences of the identities are discovered. We will analyze typical theories and how they fare.

The possibility of cyclic graphs is a core sticking point for how certain theories fare when used in the equality saturation process.

7.2.1 A quick analysis

For the theories C, I, and V, every subterm of one side of the identity is also a subterm of the other, and so no new E-classes are ever added as a result of these theories; all merges are proper.

For U and its one-sided subtheories, it is possible the subterm e is not part of the graph and needs to be seeded, but once seeded, the class of e will remain in the graph ensuring that all subsequent merges are proper.

The prime examples of 'difficult' identities that we will cover here are, then, associativity and distributivity.

7.2.2 Inverse

Inverse is something of an interesting case. Given the identity $\forall x. f(i(x), x) = e$, x does not appear at all on the right-hand side. Given that, an E-graph representing e could be extended to contain *any* new term for x as a result of this identity. However, it would seem unproductive to instantiate x with random terms, and indeed we can reason as follows:

Consider all the proofs that could involve the use of the identity for inverse in this direction. We show that $f(i(s), s) = e$ is only useful for certain values of s:

1. We could use transitivity via $t = f(i(s), s)$ and $f(i(s), s) = e$. In that case, we require already to establish an equation containing $f(i(s), s)$.

2. We could use transitivity via $f(i(s), s) = e$ and $e = t$, proving that $f(i(s), s) = t$. The usefulness of this equation can be analyzed by following this argument recursively.

3. We could use symmetry to establish $e = f(i(s), s)$, but this is again only useful in conjunction with another use of this reversed equality

4. We could use congruence to establish that, if $i(s) = t_1$ and $s = t_2$, then $f(i(s), s) = f(t_1, t_2)$.

A similar argument holds for the symmetric identity $\forall x. f(x, i(x)) = e$ in the theory N.

Such reasoning is fairly ad-hoc, and can be subsumed by analyses involving critical pairs FiXme: forward ref. This notion is useful for understanding the behaviour of even plain equality saturation, at least when dealing with identities that don't contain every variable on both sides.

7.2.3 Weak term acyclicity

[13] gives a definition of when a theory[1] is 'weakly term acyclic', and proves that such theories can be solved with equality saturation in polynomial time. This is a generalization of our basic syntactic analysis of the identities to identify which E-nodes can potentially be created by saturating under an identity.

7.3 The word problem

Doing equality saturation on a theory T, or coming up with a notion of EM(T), connects to the *word problem*.

The word problem is as follows: given some theory T and some equations, does a particular (ground) equation $t_1 = t_2$ hold?

Restricting the set of identities, we get the word problem for commutative semigroups (when T = AC), the word problem for monoids (when T = AU), etc.

7.3.1 Reductions

We can reduce the word problem to equality saturation. If we have some input equations, we can add them to an E-graph, saturate under the identities in T,

[1] actually, they define saturation under a set of rewrite rules, rather than equations, but we can treat equations as bidirectional rewrite rules

and then check if t_1 and t_2 are represented in the E-graph by the same E-class. Therefore, equality saturation is 'word-problem hard'.

We can reduce equality saturation to the word problem. If we want to determine if two terms t_1 and t_2 would be equal under saturation under T with the input equations, we can solve the word problem with those terms, those input equations, and T.

Note that this reduction is merely for the decision problem of whether two terms end up assigned to equal E-classes, rather than for reconstructing the actual (potentially infinite!) E-graph that is the fixpoint of saturation.

Since EMTs are trying to solve essentially the same problem as E-graphs with equality saturation, just in a more efficient way, any good notion of EM(T) for a theory T is essentially equivalent to a solver for the word problem over T.

7.3.2 Results on various theories

For A, the theory of semigroups, the word problem is undecidable [9], and this extends to AU, the theory of monoids.

Therefore, while EM(A) and EM(AU) might be worthwhile notions to explore, there is no terminating "A-completion" that can be applied: we can provide infrastructure to find and resolve critical pairs incrementally but never have a guarantee that process will terminate.

Adding commutativity is the key ingredient that makes the word problem decidable. The word problem for AC and ACU (commutative semigroups and commutative monoids) is ESPACE-complete [7], which provides a fairly tight bound on the worst-case complexity.

For distributivity, the word problem is again undecidable unless the distributing operator (the 'multiplication') is also commutative, in which case Buchberger-like algorithms can decide it. FiXme: source

FiXme: What about idempotency?

Introducing negation, giving us the theory ACUN of abelian groups, moves the bound from ESPACE to polynomial-time: Buchberger's algorithm reduces to finding the Hermite normal form of an integer matrix FiXme: cite. If we have a distributive operator where we can *divide*, such as the theory of fields or vector spaces, then we can use Gaussian elimination, which is not just polynomial-time but subcubic FiXme: cite

7.4 On the general notion of EMT

So, for a theory T, we need to build in to our E-graph a system equivalent in strength to a word problem solver (even if, as for $T = \mathrm{A}$, that system is partial).

7.4.1 Critical pairs

Resolving T-critical pairs is a natural way to integrate such a solver. If we have a rewriting relation $\to_{\mathcal{R}}$, then a **peak** is a trio of terms s_1, s_2 and t such that $s_1 \leftarrow_{\mathcal{R}} t \to_{\mathcal{R}} s_2$. A **critical pair** is a peak that cannot be obtained as a non-trivial substitution instance of another. For instance, if we have the rules $f(x, y) \to_{\mathcal{R}} g(y, y, x)$ and $h(x) \to_{\mathcal{R}} x$, then the term $f(h(g(x, y, z)), w)$ is a peak, because

$$g(w, w, h(g(x, y, z))) \leftarrow_{\mathcal{R}} f(h(g(x, y, z)), w) \to_{\mathcal{R}} f(g(x, y, z), w)$$

Now, as we have seen, E-graphs take a set of equations E and build a rewrite system $\to_E$ such that $s =_E t$ iff there is a c such that $s \to_E^* c \leftarrow_E^* t$. If we have a rewriting system with a peak $s_1 \leftarrow_{\mathcal{R}} t \to_{\mathcal{R}} s_2$ and we want to make $\mathcal{R}$ be able to describe a theory of equality, we need to find some way to enforce **confluence**: because s_1 and s_2 ought to be equal if $\mathcal{R}$ is taken to be axiomatizing equalities, we need to ensure there is a term c such that $s_1 \to_{\mathcal{R}}^* c \leftarrow_{\mathcal{R}}^* s_2$. Ground Knuth-Bendix completion FiXme: cite is one method to do this, based on adding a new rewrite $s_1 \to_{\mathcal{R}} s_2$ to 'orient' each critical pair.

E-graphs take another approach: a nonconfluent critical pair $s_1 \leftarrow t \to s_2$ can only arise if there are rewrite rules that *overlap*, which we can put into two categories: if $f(\ldots) \to s_1$ and $f(\ldots) \to s_2$ can both apply to the same ground term, this will manifest in an E-graph as an E-node that is part of two E-classes, which will be merged on rebuilding. If $f(\ldots, g, \ldots) \to s_1$ and $g \to s_2$, then note that we will not actually represent the term $f(\ldots, g, \ldots)$ directly: rather, we will introduce a new name e_1 and write it as $f(\ldots, e_1, \ldots) \to s_1$. In this case, after taking into account the rewrite applied to g, the critical pair is resolved automatically, as if $g = s_2 = e_1$ then $f(\ldots, g, \ldots) = f(\ldots, s_2, \ldots) = f(\ldots, e_1, \ldots) = s_1$ will all follow from the presented equational theory.

So, the flattening that happens when building E-graphs can be seen as a process of introducing new names for every subterm, since every subterm is a *potential* critical pair. By introducing these names, we restrict the need to scan for critical pairs to the case of complete overlap: two equal terms in different E-classes.

FiXme: This section in general includes bits and pieces that I now think have been explained well earlier. E.g.:

Based on this, we can diagnose the problems of E-graphs on AC as the E-graph speculatively adding all 'AC-subterms' to the E-graph as any one of them

could part of some critical pair, up to AC. For instance, if we have the terms $a+b+c+d$ and $c+e+b$, they have an overlap in the AC-subterm $b+c$, and if these terms were assigned to different E-classes, we would have the AC-critical pair $e_1 + e \leftarrow a+b+c+d+e \rightarrow a+d+e_2$.

So, while E-graphs are built around presenting a confluent ground rewriting system, the notion of EMT over a theory T suggests moving to the notion of confluent ground T-rewriting system in the natural way, in which case we need to pay special attention to critical pairs, which need to be resolved to maintain confluence.

Chapter 8

Prior work

8.1 EMTs: prior attempts

That *some* notion of EMT ought to exist has been stated multiple times before ([16], [10], [11]). The hope is to be able to use theory-specific knowledge to integrate certain identities directly into the procedures, rather than having to time-consumingly match, seed, and merge as directed by the identities. We will briefly go over the key intutition behind these proposals, and show that they fail to take into account the T-critical pair idea we outlined above.

8.1.1 Intuition 1: Canonizers

Zucker [16] notes that we have *canonizers* for many theories, which provide a way to solve the problem of ground equality modulo theories. A canonizer for a theory T is just some function on terms c such that, if $t_1 =_T t_2$, then $c(t_1) = c(t_2)$: that is, it shifts the problem from equality modulo T to just ordinary term equality.

8.1.2 Intutition 2: Containers

The signature for an E-graph, Σ, is equivalent to a certain kind of *container*: one that has a number of 'shapes' (corresponding to the function symbols) and each shape has an ordered collection of slots (with length given by the arity of the shape). Thus, it is natural to consider generalizing this to containers of more general varieties. If one assigns to a single symbol one shape of every arity , with ordered slots, this gives the theory of AU: terms like $+()$, $+(a, b)$, $+(a, b, c)$ are all covered by this notion, and one doesn't need to arbitarily split a given list into binary uses of $+(-, -)$ and nilary uses of e. If the ordering constraint is dropped on the collections of slots, we get multiset containers, suitable for

ACU, and moving to set containers gives something akin to ACIU – ACU with idempotence.

The second intuition also arises naturally out of concrete implementation work on E-graphs. Given a signature with an ACU binary operator f with unit e, and some other operators g, h, we can imagine building a parameterized data structure for term-fragments along the lines of the following pseudo-Haskell:

```
data TermF a
        = E
        | F a a
        | G a a a
        | H a
```

Looking at this structure and considering the ACU nature of f and e, it is only natural to change this to

```
data TermF a
        = Fs (Multiset a)
        | G a a a
        | H a
```

for an appropriate type constructor of multisets.

FiXme: Cite literature on polynomial functors/containers?

8.1.3 Canonizers and containers and the problem of flattening

The first intuition naturally connects to the second intuition if we consider the generalized idea of 'container' as representing some canonical form. Both approaches at first glance seem to take care of a lot of potential fiddling with identities in such theories. However, there is one issue: while we manage to remove the some part of the theory with this approach, there is still one derived identity which we need to handle.

For instance, consider an AU pair of operators $+, 0$. The identities are

$$a + (b + c) = (a + b) + c \tag{8.1}$$
$$a + 0 = a \tag{8.2}$$
$$0 + a = a \tag{8.3}$$

Using a list representation, these become

$$[a, [b, c]] = [[a, b], c] \tag{8.4}$$
$$[a, []] = a \tag{8.5}$$
$$[[], a] = a \tag{8.6}$$

These identities are not entirely automatic just by switching to lists: we need the additional embedding and and flattening laws

$$a = [a] \tag{8.7}$$
$$[[a_1, \ldots, a_n], [b_1, \ldots, b_m], \ldots] = [a_1, \ldots, a_n, b_1, \ldots, b_m, \ldots] \tag{8.8}$$

Or in other words, every term can be embedded in a list of just that term, and a list of lists can be flattened by concatenating all of its elements.

Eq 8.7 and eq 8.8 together with the basic rules of lists are what is required to prove eq 8.4, eq 8.5, and eq 8.6.

FiXme: Cite literature on unbiased presentations? This is a monad law...

It is the need to handle embedding and flattening that gives rise to problems with an approach based on just canonizers or containers. We will then also see that embedding and flattening are the two rules that govern the formation of T-critical pairs.

A typical example of where these approaches break down is given by

$$a + b = b \tag{8.9}$$
$$a + a = c \tag{8.10}$$
$$c + b = d \tag{8.11}$$

taken modulo associativity of $+$.

The goal is to prove simply that $b = d$, which follows from the simple deduction

$$
\begin{aligned}
& b \\
& = a + b \\
& = a + (a + b) \\
& = (a + a) + b \\
& = c + b \\
& = d
\end{aligned}
$$

If we try the containers approach, we start by building up an E-graph with the following E-classes:

$e_1 = \{b, [a, b]\}, e_2 = \{c, [a, a]\}, e_3 = \{d, [c, b]\}$.

However, this is by itself, entirely inert. We need to use flattening to reason that, since $b = [a, b]$, that $[a, b] = [a, [a, b]] = [a, a, b]$. Then we need to use the converse of flattening to notice that $[a, a, b] = [[a, a], b] = [c, b] = d$.

For the canonizer approach, we can easily canonize the input equations: indeed, all of them have both sides in canonical form already. What is missing, then? Well, if we view the E-graph as a rewrite system, there is of course an A-critical pair between the rewrite $a+b \to e_1$ and $a+a \to e_2$. The critical pair is $a+a+b$, which can be rewritten as either $a + e_1$ or $e_2 + b$.

We conclude that canonizers and containers are not enough: they can rewrite terms that might appear in equations to a canonical form, but this is not enough to make the equations $s_i = t_i$ when flattened and oriented into the form $s_i \to^* e_j \leftarrow^* t_i$ to naturally form a T-confluent system rewrite.

There are some cases where canonizers or containers avoid this problem: for instance, consider the theory C. A critical pair mod C can only be of the form $e_1 \leftarrow a+b =_C b+a \to e_2$. If we always canonize (mod C) both sides of equations when we orient them (say, by sorting according to some stable order), then we will have a confluent system mod C. (This requires some care to maintain the ordering given that a and b might be E-class ids and subject to change via union-find and rebuilding).

FiXme: Backward ref to minimally expansive theories, to be renamed to weak acyclicity or whatever. Actually, conjecture: all minimally expansive/weakly acyclicity theories can be done with just a canonizer/container

8.2 Congruence closure modulo theories

There has been work on congruence closure modulo various theories:

- [1] describe a congruence-closure modulo AC as building a rewriting system over an extended signature, and provide an algorithm for converting this back to a rewriting system over the original signature

- [14] provides an in-depth description of congruence-closure modulo AC and various other variants that can be covered by Buchberger's algorithm (polynomials, rings, etc.)

- [5] describes a congruence-closure modulo AC algorithm, and [6] extends this in a modular way to handle additional identities for identities including combinations of idempotency, nilpotency, units, and cancelativity

- [3] gives a congruence-closure modulo a 'solvable' theory. Unlike various modulo-AC variants, this requires the ability to, for instance, 'solve' the equation $3x = 2y$ by turning it into $x = \frac{2}{3}y$, which requires inverses for the relevant operators.

Chapter 9

Conclusion and future work

EMTs provide a good basis for integrating certain common well-behaved theories more tightly into the core of E-graphs, rather than allowing the potentially explosive equality saturation to handle them. However, there are still fundamental limitatations to how 'nice' EMTs can be: they expand the E-graph during the resolution of critical pairs, unlike congruence-closure which can only shrink it, and the possible performance gains are bounded tightly by the complexity of solving the word problems for the relevant theories.

9.1 Future work: EMTs for other theories

Given that there are already several variants of congruence-closure modulo theories (Section 8.2), it would be useful to evaluate EMTs on these theories, including with E-matching modulo these theories. Many of them require Buchberger-like algorithms similar to the one we have presented for AC. Variations of this algorithm for certain theories (Section 7.3.2) can be polynomial-time rather than ESpace.

9.2 Future work: improving AC-closure

Buchberger's algorithm has the F_4 and F_5 variants and an extensive literature on optimizing it. Importing these variants into an EMT implementation could provide significant speedup. Also, as discussed in Section 6.2.4, there is the potential to make the AC-closure more incremental and reduce the cost associated with computing it.

9.3 Future work: constrained matching

FiXme: I can probably find a precedent for this

As mentioned in Section 5.4.3, adding constraints to patterns for matching could significantly cut down the search space for matching modulo theories when the full space of matches, which includes a lot of redundant symmetries, is not needed.

Bibliography

[1] L. Bachmair et al. "Congruence Closure Modulo Associativity and Commutativity". In: *Frontiers of Combining Systems*. Ed. by Hélène Kirchner and Christophe Ringeissen. Red. by Gerhard Goos, Juris Hartmanis, and Jan Van Leeuwen. Vol. 1794. Series Title: Lecture Notes in Computer Science. Berlin, Heidelberg: Springer Berlin Heidelberg, 2000, pp. 245–259. ISBN: 978-3-540-67281-4 978-3-540-46421-1. DOI: 10.1007/10720084_16. URL: http://link.springer.com/10.1007/10720084_16 (visited on 02/24/2026).

[2] Leo Bachmair and Ashish Tiwari. "Abstract Congruence Closure and Specializations". In: *Automated Deduction - CADE-17*. Ed. by David McAllester. Berlin, Heidelberg: Springer, 2000, pp. 64–78. ISBN: 978-3-540-45101-3. DOI: 10.1007/10721959_5.

[3] Sylvain Conchon et al. "CC(X): Semantic Combination of Congruence Closure with Solvable Theories". In: *Electronic Notes in Theoretical Computer Science*. Proceedings of the 5th International Workshop on Satisfiability Modulo Theories (SMT 2007) 198.2 (May 6, 2008), pp. 51–69. ISSN: 1571-0661. DOI: 10.1016/j.entcs.2008.04.080. URL: https://www.sciencedirect.com/science/article/pii/S1571066108002958 (visited on 02/08/2026).

[4] Peter J. Downey, Ravi Sethi, and Robert Endre Tarjan. "Variations on the Common Subexpression Problem". In: *Journal of the ACM* 27.4 (Oct. 1980), pp. 758–771. ISSN: 0004-5411, 1557-735X. DOI: 10.1145/322217.322228. URL: https://dl.acm.org/doi/10.1145/322217.322228 (visited on 02/08/2026).

[5] Deepak Kapur. "A Modular Associative Commutative (AC) Congruence Closure Algorithm". In: *LIPIcs, Volume 195, FSCD 2021* 195 (2021). Ed. by Naoki Kobayashi. Artwork Size: 21 pages, 711938 bytes ISBN: 9783959771917 Medium: application/pdf, 15:1–15:21. ISSN: 1868-8969. DOI: 10.4230/LIPICS.FSCD.2021.15. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.FSCD.2021.15 (visited on 03/17/2026).

[6] Deepak Kapur. "Modularity and Combination of Associative Commutative Congruence Closure Algorithms enriched with Semantic Properties". In: *Logical Methods in Computer Science* Volume 19, Issue 1 (Mar. 14, 2023), p. 8693. ISSN: 1860-5974. DOI: 10.46298/lmcs-19(1:19)2023. URL: https://lmcs.episciences.org/8693 (visited on 02/24/2026).

[7] Ernst W Mayr and Albert R Meyer. "The complexity of the word problems for commutative semigroups and polynomial ideals". In: *Advances in Mathematics* 46.3 (Dec. 1, 1982), pp. 305–329. ISSN: 0001-8708. DOI: 10.1016/0001-8708(82)90048-2. URL: https://www.sciencedirect.com/science/article/pii/0001870882900482 (visited on 03/11/2026).

[8] Greg Nelson and Derek C. Oppen. "Fast Decision Procedures Based on Congruence Closure". In: *Journal of the ACM* 27.2 (Apr. 1980), pp. 356–364. ISSN: 0004-5411, 1557-735X. DOI: 10.1145/322186.322198. URL: https://dl.acm.org/doi/10.1145/322186.322198 (visited on 02/08/2026).

[9] Carl-Fredrik Nyberg-Brodda. *G. S. Tseytin's seven-relation semigroup with undecidable word problem*. Jan. 22, 2024. DOI: 10.48550/arXiv.2401.11757. arXiv: 2401.11757[math]. URL: http://arxiv.org/abs/2401.11757 (visited on 02/27/2026).

[10] Pavel Panchekha. *LIN-graphs, an Egraph modulo T*. Pavel's Blog. May 23, 2025. URL: https://pavpanchekha.com/blog/lin-graphs.html.

[11] Saul Shanabrook. *Custom Data Structures in E-Graphs*. PLSE Blog. Feb. 24, 2026. URL: https://uwplse.org/2026/02/24/egglog-containers.html.

[12] Natarajan Shankar. "Little Engines of Proof". In: *FME 2002:Formal Methods—Getting IT Right*. Ed. by Lars-Henrik Eriksson and Peter Alexander Lindsay. Red. by Gerhard Goos, Juris Hartmanis, and Jan Van Leeuwen. Vol. 2391. Series Title: Lecture Notes in Computer Science. Berlin, Heidelberg: Springer Berlin Heidelberg, 2002, pp. 1–20. ISBN: 978-3-540-43928-8 978-3-540-45614-8. DOI: 10.1007/3-540-45614-7_1. URL: http://link.springer.com/10.1007/3-540-45614-7_1 (visited on 03/17/2026).

[13] Dan Suciu, Yisu Remy Wang, and Yihong Zhang. *Semantic foundations of equality saturation*. Jan. 5, 2025. DOI: 10.48550/arXiv.2501.02413. arXiv: 2501.02413[cs]. URL: http://arxiv.org/abs/2501.02413 (visited on 02/04/2026).

[14] Ashish Tiwari. *Decision procedures in automated deduction*. State University of New York at Stony Brook, 2000. URL: https://search.proquest.com/openview/30046982652b13cadc7b85d653cdf547/1?pq-origsite=gscholar&cbl=18750&diss=y (visited on 03/18/2026).

[15] Max Willsey et al. "egg: Fast and extensible equality saturation". In: *Proceedings of the ACM on Programming Languages* 5 (POPL Jan. 4, 2021), pp. 1–29. ISSN: 2475-1421. DOI: 10.1145/3434304. URL: https://dl.acm.org/doi/10.1145/3434304 (visited on 02/08/2026).

[16] Philip Zucker. *Omelets Need Onions: E-graphs Modulo Theories via Bottom-up E-matching*. Apr. 19, 2025. DOI: 10.48550/arXiv.2504.14340. arXiv: 2504.14340[cs]. URL: http://arxiv.org/abs/2504.14340 (visited on 01/20/2026).